LabX ADV: Advanced Web Scraping & Introduction to Automated Agents

This week, we push beyond basic web scraping to explore more advanced techniques, including handling dynamic web content and introducing the foundational concepts of "agentic AI" in automation. You will learn to build a more robust and adaptable scraper that can follow instructions from a configuration, simulating an autonomous agent.

Learning Outcome/Trait Assessment:

- Master advanced web scraping techniques, particularly for dynamic, JavaScript-rendered content using Selenium.
- Implement robust error handling, retry mechanisms, and basic anti-blocking strategies.
- Design and utilize configuration files to drive scraping behavior, mimicking an "agentic" approach.
- Understand data structuring for scraped information and persist it in various formats.
- Analyze complex web page structures and write effective CSS selectors or XPath expressions.
- Comprehend the ethical and legal responsibilities associated with advanced automated scraping.
- **Trait Assessment:** Advanced problem-solving, system design, robustness engineering, configuration management, ethical decision-making in automation, foundational AI thinking.

Detailed Lecture Topics:

1. **Recap & Limitations of Basic Scraping:**
    - Review requests + BeautifulSoup for static HTML.
    - Limitations: What happens with JavaScript-loaded content, infinite scrolling, button clicks?
2. **Dynamic Content Scraping with Selenium:**
    - **Introduction to Selenium:** What it is, how it works (browser automation), WebDriver concept.
    - **Setup:** Installing selenium (pip install selenium), downloading and configuring browser-specific WebDrivers (e.g., ChromeDriver, GeckoDriver).
    - **Basic Browser Control:** Launching browsers, navigating URLs (driver.get()), closing.
    - **Finding Elements:** find_element(By.ID, "id"), find_element(By.CSS_SELECTOR, "css selector"), find_element(By.XPATH, "xpath").
    - **Interacting with Elements:** Clicking buttons (.click()), typing text (.send_keys()), getting text (.text).

    - **Waiting Strategies:**
        - Implicit Waits (driver.implicitly_wait()): A global timeout for finding elements.
        - Explicit Waits (WebDriverWait, EC): Waiting for specific conditions (e.g., element to be clickable, visible). Crucial for dynamic content.
    - **Headless Browsers:** Running browsers without a UI for efficiency (--headless argument).
3. **Robustness & Anti-Blocking Strategies:**
    - **Delays:** Using time.sleep() for simple delays (basic), but emphasizing explicit waits for better reliability.
    - **User-Agent Spoofing:** Setting custom User-Agents in Selenium options.
    - **Basic Retry Mechanisms:** Implementing loops with try-except blocks for transient errors.
    - **IP Rotation (Conceptual):** Brief mention of proxies and VPNs (complex for this level).
4. **Data Structuring & Storage:**
    - Defining Python classes or dictionaries to represent scraped entities (e.g., Product with name, price, description).
    - Saving structured data to CSV files (using csv module) and JSON files (using json module).
5. **Introduction to "Agentic" Scraping Concepts:**
    - **What is an Agent?** Definition: Goal-oriented, perceives environment, takes actions, adapts.
    - **Configuration-Driven Agents:** Simulating agentic behavior by providing structured rules (a "plan") in a configuration file.
    - **Defining Scraping "Rules" (.json config):**
        - start_url
        - pagination_selector (CSS/XPath for "next page" button/link)
        - item_container_selector (Selector for a single product/article block)
        - item_data_selectors (Dictionary of selectors for specific fields: name_selector, price_selector, description_selector).
    - **Automation Loop:** The agent (your scraper) reads the config, navigates, extracts data, finds the next page, and repeats until no more pages.
6. **Ethical and Legal Deep Dive for Advanced Scraping:**
    - Re-emphasize robots.txt and Terms of Service.
    - Risks of IP blocking, legal action (Cease and Desist letters).
    - Importance of rate limiting and respectful scraping.
    - How Selenium might be seen differently by website owners.

Prototype: Config-Driven Product Scraper Agent

We will build an "Agentic Product Scraper" that reads a configuration file to scrape product

data from a (hypothetical or very simple public) e-commerce listing page, handle pagination, and save the data.

Note: For demonstration purposes, we will assume a simple target website structure. Using a live, complex commercial site can lead to blocks or legal issues. A mock site, or a very basic public directory page, is recommended for student exercises. We'll use a *conceptual* example_site_config.json that *describes* a typical e-commerce page structure.

Goals of the Agent:

1. Read a site-specific configuration.
2. Navigate to the starting URL.
3. Extract predefined product details from the current page.
4. Find and click the "next page" button/link.
5. Repeat until no more pages are found or a limit is reached.
6. Save all extracted data to a JSON file.

Code Base Structure (GitHub Ready)

```
agentic-web-scraper/
├── src/
│   ├── __init__.py        # Makes 'src' a Python package
│   ├── scraper_agent.py    # Main agent logic (reads config, controls selenium)
│   ├── config_parser.py    # Handles reading and validating scraping configurations
│   ├── data_models.py      # Python classes/dataclasses for scraped data (e.g., Product)
│   ├── utils.py            # Helper functions (e.g., waits, error handling)
│   └── driver_manager.py   # Manages Selenium WebDriver setup (NEW)
├── configs/
│   └── example_site_config.json # JSON config for a specific site
├── data/
│   └── scraped_products.json    # Output data
├── main.py
├── .gitignore
├── README.md
└── docs/
    └── ETHICS.md           # Dedicated ethics/legal considerations document
```

Full Source Code (Implemented Prototype)

1. agentic-web-scraper/src/__init__.py

Python

```
# agentic-web-scraper/src/__init__.py
# This file indicates that the 'src' directory is a Python package.
```

2. agentic-web-scraper/src/driver_manager.py

Python

```
# agentic-web-scraper/src/driver_manager.py
import os
from selenium import webdriver
from selenium.webdriver.chrome.service import Service as ChromeService
from selenium.webdriver.firefox.service import Service as FirefoxService
from webdriver_manager.chrome import ChromeDriverManager
from webdriver_manager.firefox import GeckoDriverManager

class DriverManager:
    """
    Manages the setup and teardown of Selenium WebDriver instances.
    Automates driver download if not present.
    """
    def __init__(self, browser_name='chrome', headless=True):
        self.browser_name = browser_name.lower()
        self.headless = headless
        self.driver = None

    def get_driver(self):
        """
        Initializes and returns a WebDriver instance.
        """
```

```
        if self.driver:
            return self.driver

        if self.browser_name == 'chrome':
            options = webdriver.ChromeOptions()
            if self.headless:
                options.add_argument('--headless')
            options.add_argument('--no-sandbox') # Required for some environments (e.g., Docker)
            options.add_argument('--disable-dev-shm-usage') # Required for some environments
            options.add_argument('--log-level=3') # Suppress console logs
            options.add_argument('user-agent=Mozilla/5.0 (Windows NT 10.0; Win64; x64)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/91.0.4472.124 Safari/537.36')

            try:
                service = ChromeService(ChromeDriverManager().install())
                self.driver = webdriver.Chrome(service=service, options=options)
            except Exception as e:
                print(f"Error setting up Chrome driver: {e}")
                print("Make sure Chrome browser is installed and try running 'pip install webdriver-manager'.")
                return None
        elif self.browser_name == 'firefox':
            options = webdriver.FirefoxOptions()
            if self.headless:
                options.add_argument('--headless')
            options.add_argument('user-agent=Mozilla/5.0 (Windows NT 10.0; Win64; rv:91.0)
Gecko/20100101 Firefox/91.0')

            try:
                service = FirefoxService(GeckoDriverManager().install())
                self.driver = webdriver.Firefox(service=service, options=options)
            except Exception as e:
                print(f"Error setting up Firefox driver: {e}")
                print("Make sure Firefox browser is installed and try running 'pip install webdriver-manager'.")
                return None
        else:
            raise ValueError(f"Unsupported browser: {self.browser_name}. Choose 'chrome' or 'firefox'.")

        # Set implicit wait time (optional, but good for dynamic pages)
        self.driver.implicitly_wait(10) # waits up to 10 seconds for elements to appear

        return self.driver
```

```
    def quit_driver(self):
        """
        Quits the WebDriver instance if it's running.
        """
        if self.driver:
            self.driver.quit()
            self.driver = None
            print("WebDriver quit successfully.")

# Note: For webdriver_manager to work, run 'pip install webdriver-manager'
```

3. agentic-web-scraper/src/config_parser.py

Python

```
# agentic-web-scraper/src/config_parser.py
import json
import os

class ConfigParser:
    """
    Parses and validates the scraping configuration from a JSON file.
    """
    def __init__(self, config_path):
        self.config_path = config_path
        self.config = {}

    def load_config(self):
        """Loads the configuration from the specified JSON file."""
        if not os.path.exists(self.config_path):
            raise FileNotFoundError(f"Configuration file not found: {self.config_path}")

        try:
            with open(self.config_path, 'r', encoding='utf-8') as f:
                self.config = json.load(f)
            self._validate_config()
            print(f"Configuration loaded successfully from {self.config_path}")
```

```
            return self.config
        except json.JSONDecodeError as e:
            raise ValueError(f"Invalid JSON format in config file: {e}")
        except Exception as e:
            raise Exception(f"An error occurred while loading config: {e}")

    def _validate_config(self):
        """
        Validates the loaded configuration to ensure required fields are present.
        This is a basic validation. More complex validation might involve regex for selectors.
        """
        required_fields = [
            "start_url",
            "pagination_selector",
            "item_container_selector",
            "item_data_selectors"
        ]

        for field in required_fields:
            if field not in self.config:
                raise ValueError(f"Missing required field in config: '{field}'")

        required_item_data_fields = ["name", "price"] # Minimum fields for a product
        for field in required_item_data_fields:
            if field not in self.config["item_data_selectors"]:
                raise ValueError(f"Missing required item data selector: '{field}'")

        # Ensure selectors are strings
        for key, value in self.config["item_data_selectors"].items():
            if not isinstance(value, str):
                raise ValueError(f"Item data selector '{key}' must be a string.")

        if not isinstance(self.config["pagination_selector"], str):
            raise ValueError("Pagination selector must be a string.")

        if not isinstance(self.config["item_container_selector"], str):
            raise ValueError("Item container selector must be a string.")

        print("Configuration validated.")
```

4. agentic-web-scraper/src/data_models.py

Python

```python
# agentic-web-scraper/src/data_models.py
from dataclasses import dataclass, asdict
import json

@dataclass
class Product:
    """
    Data model for a scraped product.
    Using dataclasses for clean and explicit data representation.
    """
    name: str
    price: str
    description: str = None
    url: str = None
    image_url: str = None
    # Add more fields as needed based on your config and target site

    def to_dict(self):
        """Converts the Product dataclass instance to a dictionary."""
        return asdict(self)

    @classmethod
    def from_dict(cls, data):
        """Creates a Product instance from a dictionary."""
        return cls(**data)

# You can add other data models here if your agent scrapes different types of entities
# For example, @dataclass class Article: ...
```

5. agentic-web-scraper/src/utils.py

Python

```python
# agentic-web-scraper/src/utils.py
import time
import json
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
from selenium.common.exceptions import TimeoutException, NoSuchElementException,
WebDriverException

def save_data_to_json(data, filename, directory='data'):
    """Saves a list of dictionaries to a JSON file."""
    os.makedirs(directory, exist_ok=True)
    filepath = os.path.join(directory, filename)
    try:
        with open(filepath, 'w', encoding='utf-8') as f:
            json.dump(data, f, indent=4, ensure_ascii=False)
        print(f"Data saved to {filepath}")
        return True
    except IOError as e:
        print(f"Error saving data to JSON file: {e}")
        return False

def wait_for_element(driver, by, value, timeout=10):
    """
    Waits for an element to be present on the DOM and visible.
    Returns the element or raises a TimeoutException.
    """
    try:
        element = WebDriverWait(driver, timeout).until(
            EC.presence_of_element_located((by, value))
        )
        # Further check if element is visible
        WebDriverWait(driver, timeout).until(
            EC.visibility_of(element)
        )
        return element
    except TimeoutException:
        print(f"Timeout waiting for element: {value}")
        return None
    except NoSuchElementException:
        print(f"Element not found after wait: {value}")
        return None
```

```python
def robust_click(driver, by, value, max_attempts=3, delay_between_attempts=2):
    """
    Attempts to click an element multiple times if it's not immediately clickable.
    """
    for attempt in range(max_attempts):
        try:
            element = wait_for_element(driver, by, value)
            if element:
                element.click()
                return True
            else:
                return False # Element not found even after waiting
        except WebDriverException as e:
            print(f"Click attempt {attempt + 1} failed: {e}")
            if attempt < max_attempts - 1:
                time.sleep(delay_between_attempts)
    print(f"Failed to click element {value} after {max_attempts} attempts.")
    return False

def scroll_to_bottom(driver):
    """Scrolls to the bottom of the page."""
    driver.execute_script("window.scrollTo(0, document.body.scrollHeight);")
    time.sleep(2) # Give some time for content to load
```

6. agentic-web-scraper/src/scraper_agent.py

Python

```python
# agentic-web-scraper/src/scraper_agent.py
import time
from selenium.webdriver.common.by import By
from selenium.common.exceptions import NoSuchElementException, TimeoutException,
StaleElementReferenceException
import logging

from .driver_manager import DriverManager
```

```
from .data_models import Product
from .utils import wait_for_element, robust_click, save_data_to_json, scroll_to_bottom

# Configure basic logging
logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s - %(message)s')

class ScraperAgent:
    """
    An agentic web scraper that operates based on a provided configuration.
    It can navigate, extract data, and handle pagination.
    """
    def __init__(self, config, browser='chrome', headless=True):
        self.config = config
        self.driver_manager = DriverManager(browser_name=browser, headless=headless)
        self.driver = None
        self.scraped_products = []
        self.max_pages = config.get("max_pages", 5) # Default to scrape max 5 pages
        logging.info(f"Scraper Agent initialized with config for {config['start_url']}")

    def _get_element_text(self, parent_element, selector, selector_type=By.CSS_SELECTOR):
        """Helper to safely get text from an element."""
        try:
            element = parent_element.find_element(selector_type, selector)
            return element.text.strip()
        except NoSuchElementException:
            # logging.warning(f"Element not found for selector: {selector}")
            return None
        except StaleElementReferenceException:
            # logging.warning(f"Stale element reference for selector: {selector}. Retrying might be needed.")
            return None # Or handle retry logic here

    def _get_element_attribute(self, parent_element, selector, attribute,
selector_type=By.CSS_SELECTOR):
        """Helper to safely get an attribute from an element."""
        try:
            element = parent_element.find_element(selector_type, selector)
            return element.get_attribute(attribute).strip()
        except NoSuchElementException:
            # logging.warning(f"Element not found for selector: {selector} (attribute: {attribute})")
            return None
        except StaleElementReferenceException:
```

```
            # logging.warning(f"Stale element reference for selector: {selector}. Retrying might be needed.")
            return None

    def _parse_product_data(self, item_element):
        """
        Extracts product data from a single product item element based on config selectors.
        """
        selectors = self.config["item_data_selectors"]

        name = self._get_element_text(item_element, selectors.get("name", ""))
        price = self._get_element_text(item_element, selectors.get("price", ""))
        description = self._get_element_text(item_element, selectors.get("description", ""),
By.CSS_SELECTOR)
        url = self._get_element_attribute(item_element, selectors.get("url", ""), "href",
By.CSS_SELECTOR)
        image_url = self._get_element_attribute(item_element, selectors.get("image_url", ""), "src",
By.CSS_SELECTOR)

        if name and price: # Basic validation for a valid product
            return Product(name=name, price=price, description=description, url=url,
image_url=image_url)
        return None

    def scrape_page(self):
        """
        Scrapes product items from the current page.
        """
        logging.info(f"Scraping page: {self.driver.current_url}")

        # Wait for product container to be present
        try:
            # Assumes item_container_selector finds the parent of all product items
            # Or directly the items if it's a direct selector for each item.
            # Using find_elements to get all items
            item_elements = self.driver.find_elements(By.CSS_SELECTOR,
self.config["item_container_selector"])
            # Fallback for XPath if CSS selector doesn't yield results
            if not item_elements:
                item_elements = self.driver.find_elements(By.XPATH,
self.config["item_container_selector"])
```

```
            logging.info(f"Found {len(item_elements)} potential product items on this page.")

            for item_element in item_elements:
                product = self._parse_product_data(item_element)
                if product:
                    self.scraped_products.append(product.to_dict())
                    # logging.debug(f"Extracted: {product.name} - {product.price}") # Use debug for verbose output
                else:
                    logging.warning("Could not parse product data from an item element.")
        except NoSuchElementException:
            logging.error(f"Item container selector '{self.config['item_container_selector']}' not found on page.")
        except Exception as e:
            logging.error(f"An error occurred while scraping items: {e}")

    def run(self):
        """
        Executes the scraping process.
        """
        self.driver = self.driver_manager.get_driver()
        if not self.driver:
            logging.error("Failed to get WebDriver. Exiting.")
            return

        try:
            self.driver.get(self.config["start_url"])
            current_page = 1

            while current_page <= self.max_pages:
                logging.info(f"Processing page {current_page} at {self.driver.current_url}")
                self.scrape_page()
                time.sleep(self.config.get("delay_between_pages", 2)) # Respectful delay

                # Check for pagination
                pagination_selector = self.config["pagination_selector"]
                if pagination_selector:
                    try:
                        # Attempt to find the next page button/link using specified selector
                        # Use wait_for_element here for robustness
                        next_page_button = wait_for_element(self.driver, By.CSS_SELECTOR,
```

```
pagination_selector, timeout=5)
                if not next_page_button:
                    next_page_button = wait_for_element(self.driver, By.XPATH, pagination_selector, timeout=5)

                if next_page_button and next_page_button.is_displayed() and next_page_button.is_enabled():
                    logging.info(f"Clicking next page: {next_page_button.text or 'No text'}")
                    robust_click(self.driver, By.CSS_SELECTOR, pagination_selector) # Use robust click
                    current_page += 1
                else:
                    logging.info("No more next page button found or not clickable. Ending pagination.")
                    break
            except (NoSuchElementException, TimeoutException):
                logging.info("Pagination element not found or timed out. Assuming last page.")
                break
            except Exception as e:
                logging.error(f"Error during pagination: {e}. Ending pagination.")
                break
        else:
            logging.info("No pagination selector configured. Scraping single page.")
            break # No pagination configured, so stop after first page

    except Exception as e:
        logging.critical(f"An unhandled error occurred during scraping: {e}")
    finally:
        self.driver_manager.quit_driver()
        logging.info(f"Scraping finished. Total products scraped: {len(self.scraped_products)}")
        save_data_to_json(self.scraped_products, "scraped_products.json")
```

7. agentic-web-scraper/main.py

Python

```
# agentic-web-scraper/main.py
import sys
import os
```

```python
import logging

# Add the 'src' directory to the Python path
sys.path.append(os.path.join(os.path.dirname(__file__), 'src'))

from config_parser import ConfigParser
from scraper_agent import ScraperAgent

# Configure logging for main script
logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s - %(message)s')

def main():
    """
    Main entry point for the Agentic Web Scraper.
    Loads config and runs the scraping agent.
    """
    config_file_path = os.path.join(os.path.dirname(__file__), 'configs', 'example_site_config.json')

    try:
        config_parser = ConfigParser(config_file_path)
        config = config_parser.load_config()

        # Choose browser ('chrome' or 'firefox') and headless mode (True/False)
        # Note: Headless mode may sometimes fail on complex sites that detect headless browsers.
        # For debugging, set headless=False to see the browser actions.
        scraper_agent = ScraperAgent(config, browser='chrome', headless=True)
        scraper_agent.run()

    except FileNotFoundError as e:
        logging.error(f"Configuration file error: {e}")
        logging.info("Please ensure 'configs/example_site_config.json' exists.")
    except ValueError as e:
        logging.error(f"Configuration validation error: {e}")
        logging.info("Please check the format and required fields in your config file.")
    except Exception as e:
        logging.critical(f"An unexpected error occurred during execution: {e}")

if __name__ == "__main__":
    main()
```

8. agentic-web-scraper/configs/example_site_config.json

JSON

```
{
  "start_url": "https://books.toscrape.com/catalogue/category/books_1/index.html",
  "max_pages": 3,
  "delay_between_pages": 2,
  "item_container_selector": "article.product_pod",
  "pagination_selector": "li.next a",
  "item_data_selectors": {
    "name": "h3 a",
    "price": "p.price_color",
    "description": null,
    "url": "h3 a",
    "image_url": "div.image_container a img"
  }
}
```

- **Explanation for example_site_config.json:**
  - start_url: The initial URL to begin scraping.
  - max_pages: How many pages to scrape before stopping.
  - delay_between_pages: A delay in seconds between navigating to new pages (crucial for polite scraping).
  - item_container_selector: A CSS selector that identifies a *single* product/item block on the page. The scraper will find all elements matching this selector and then extract data from within each.
  - pagination_selector: A CSS selector for the "next page" button or link. The scraper will try to click this to go to the next page.
  - item_data_selectors: A dictionary where keys are the desired data fields (e.g., "name", "price") and values are CSS selectors *relative to the item_container_selector* that locate the specific data point. null means that field is not expected on the page.

9. agentic-web-scraper/.gitignore

ข้อมูลโค้ด

```
# Python
__pycache__/
*.pyc
*.pyo
*.pyd
.Python/
env/
venv/
.ipynb_checkpoints/ # For Jupyter/Colab notebooks
.vscode/          # VS Code workspace settings if you don't want to share

# Environment
.env
# Pipenv
.venv/
# virtualenv
venv/
env/

# Data files (scraped output)
data/

# WebDriver logs
geckodriver.log
chromedriver.log
```

10. agentic-web-scraper/README.md

```
Markdown
```

```
# Week 14: Advanced Web Scraping & Introduction to Automated Agents

This project builds a more advanced, "agentic" web scraper using `Selenium` for dynamic content and a configuration-driven approach for adaptability. It demonstrates how to automate Browse, extract structured data, handle pagination, and manage data persistence.

## Key Concepts Demonstrated
```

* **Selenium for Dynamic Content**: Using a real browser (headless or visible) to interact with JavaScript-rendered web pages.
* **WebDriver Management**: Automated setup of browser drivers using `webdriver_manager`.
* **Config-Driven Automation**: The scraper's behavior (what to scrape, how to paginate) is defined in an external JSON configuration file, allowing it to adapt to different websites without code changes.
* **Modular Agent Design**: Breaking down the scraping logic into specialized modules (`scraper_agent`, `config_parser`, `data_models`, `utils`, `driver_manager`).
* **Robustness**: Implementing explicit waits, retry mechanisms for clicks, and basic error handling for network/element issues.
* **Structured Data**: Parsing extracted information into Python dataclasses and saving it to JSON.

Ethical and Legal Considerations

It is crucial to understand and adhere to ethical and legal guidelines when web scraping:

* **`robots.txt`**: Always check a website's `robots.txt` file (e.g., `https://example.com/robots.txt`) before scraping. This file specifies which parts of the site can be crawled by bots.
* **Terms of Service (ToS)**: Review the website's Terms of Service. Many sites explicitly forbid scraping.
* **Rate Limiting**: Do not overwhelm the website's server with too many requests too quickly. Use `time.sleep()` or built-in Selenium waits (as done in `scraper_agent.py`) to introduce delays.
* **Data Usage**: Be aware of intellectual property rights. You generally cannot republish scraped content without permission.
* **Legality**: Web scraping legality varies by jurisdiction and case. Always err on the side of caution. **This project is for educational purposes only.**

Setup & How to Run

1. **Clone the repository:**
   ```bash
   git clone
[https://github.com/YOUR_USERNAME/agentic-web-scraper.git](https://github.com/YOUR_USERNAME/agentic-web-scraper.git)
   cd agentic-web-scraper
   ```
2. **Install Dependencies:**
   It's highly recommended to use a virtual environment:
   ```bash
   python -m venv venv
   ```

```
   source venv/bin/activate  # On Windows: venv\Scripts\activate.bat
   pip install selenium beautifulsoup4 webdriver-manager
   ```
3. **Ensure Browser and WebDriver are compatible:**
   * The `driver_manager.py` uses `webdriver_manager` to automatically download the correct WebDriver.
   * Make sure you have either **Google Chrome** or **Mozilla Firefox** installed on your system, as these are the browsers `Selenium` will control.
4. **Configure `configs/example_site_config.json`:**
   * This file dictates what and how the scraper will extract. The provided example is configured for `books.toscrape.com`. You can modify it to target other simple sites (be respectful!).
   * **Crucially, adjust selectors based on the target website's HTML structure.** Use your browser's Developer Tools (F12) to inspect elements.
5. **Run the scraper:**
   ```bash
   python main.py
   ```
   The scraper will launch a browser (headless by default), navigate the pages, extract data, and save it to `data/scraped_products.json`.

Project Structure

```
agentic-web-scraper/
├── src/
│   ├── init.py # Python package marker
│   ├── scraper_agent.py # Main logic: reads config, navigates, extracts, paginates
│   ├── config_parser.py # Loads and validates JSON configurations
│   ├── data_models.py # Python classes for structured scraped data
│   ├── utils.py # Helper functions (save data, waits, robust clicks)
│   └── driver_manager.py # Handles Selenium WebDriver setup and teardown
├── configs/
│   └── example_site_config.json # Site-specific scraping rules
├── data/
│   └── scraped_products.json # Output file for scraped data
├── main.py # Application entry point
├── .gitignore # Files/folders to ignore in Git
├── README.md # This project overview
└── docs/
└── ETHICS.md # Important document on ethical & legal scraping
```

Debugging Advanced Web Scraping

* **WebDriver Errors**: `WebDriverException`, `SessionNotCreatedException`. Often means the WebDriver is not found, or its version is incompatible with your browser. Ensure `webdriver-manager` is installed and updated, and your browser is up-to-date.
* **Element Not Found (`NoSuchElementException`)**: Your CSS selector or XPath is incorrect, or the element hasn't loaded yet.
  * **Solution**: Use browser DevTools (F12) to re-verify selectors. Implement explicit waits (`WebDriverWait`) to ensure elements are present and visible before trying to interact.
* **Timeout Errors (`TimeoutException`)**: The element you are waiting for did not appear within the specified time.
  * **Solution**: Increase wait times, or inspect the website's loading behavior. Is it genuinely slow, or is the element truly not appearing?
* **Stale Element Reference (`StaleElementReferenceException`)**: An element you previously found is no longer attached to the DOM (e.g., after a page refresh or dynamic update).
  * **Solution**: Re-find the element after actions that might change the DOM (like a page load or AJAX update).
* **Infinite Pagination Loops**: The "next page" selector always finds something, even on the last page, or clicks the wrong element.
  * **Solution**: Carefully inspect the "next page" button on the last page. Does its selector change? Does it become disabled? Implement checks for these conditions.
* **Anti-Bot Measures**: Websites may detect automated access and block you (e.g., CAPTCHAs, IP bans).
  * **Solution**: Start with slow delays (`time.sleep()`), use realistic User-Agents, consider rotating IPs (advanced topic). For this week, prioritize understanding the mechanism over bypassing sophisticated defenses.
* **Headless vs. Headed**: If you encounter issues in headless mode, try running with `headless=False` in `main.py` to visually debug what the browser is doing.

Extension Ideas (Future Work)

* **More Robust Anti-Blocking**: Integrate proxy rotation, solve simple CAPTCHAs (using external services), or implement more sophisticated User-Agent rotation strategies.
* **Data Pipelines**: Integrate with databases (SQLite, PostgreSQL), data lakes, or cloud storage.
* **Error Reporting & Monitoring**: Implement detailed logging, email notifications for failures, or dashboarding.
* **Concurrent Scraping**: Use `threading` or `asyncio` to scrape multiple pages or items concurrently (with careful rate limiting).
* **Advanced AI Integration**:

    * **LLM-driven Selector Generation**: Use an LLM to "understand" the page structure and *generate* CSS/XPath selectors based on natural language descriptions (e.g., "get the price of the main product").
    * **Adaptive Navigation**: An LLM could decide *where to click next* based on a high-level goal and observed page content.
    * **Failure Recovery**: AI could analyze scraping failures (e.g., element not found) and suggest alternative selectors or actions.
* **GUI for Configuration**: Build a simple GUI that allows users to create and manage scraping configurations visually.
* **Scheduled Scraping**: Integrate with tools like `cron` (Linux) or Windows Task Scheduler to run the scraper automatically at intervals.

---
11. `agentic-web-scraper/docs/ETHICS.md`

```markdown
# Web Scraping Ethics and Legality Guidelines

This document outlines important ethical and legal considerations when engaging in web scraping. It is crucial to understand and adhere to these principles to avoid legal issues, IP blocking, or causing harm to website owners.

## 1. Respect `robots.txt`

The `robots.txt` file is a standard that websites use to communicate with web crawlers and scrapers. It tells bots which parts of the site they are allowed to access and which parts are forbidden.

* **Always check:** Before scraping any website, visit `https://[website-domain]/robots.txt` (e.g., `https://example.com/robots.txt`).
* **Adhere to rules:** If `robots.txt` disallows scraping a certain path, **do not scrape it**. This is a direct request from the website owner.

## 2. Review Terms of Service (ToS)

Many websites include clauses in their Terms of Service that explicitly prohibit web scraping, data mining, or automated access.

* **Read the fine print:** If a website's ToS explicitly forbids scraping, then scraping that site is a breach of contract and could lead to legal action (e.g., Cease and Desist letters, lawsuits).
* **Implied consent:** Absence of a specific prohibition does not automatically grant
```

permission.

3. Be Polite: Rate Limiting and Delays

Overwhelming a website's server with too many requests in a short period can be interpreted as a Denial-of-Service (DoS) attack, impacting the site's performance for legitimate users.

* **Introduce Delays:** Use `time.sleep()` between requests or pages to mimic human Browse behavior. A delay of 2-5 seconds is a common starting point, but adjust based on the site's load.
* **Avoid Concurrent Requests:** Unless you know what you are doing and have explicit permission, avoid sending multiple requests simultaneously.
* **Monitor Server Load:** If you notice slower response times or errors, reduce your request rate.

4. Identify Yourself (User-Agent)

When making requests, include a descriptive `User-Agent` header that identifies your scraper. This allows website administrators to contact you if there are issues.

* **Example:** `Mozilla/5.0 (compatible; MyCustomScraper/1.0; mailto:your_email@example.com)`
* **Avoid generic agents:** Do not impersonate common browsers or Googlebot unless your scraper genuinely behaves like them and respects their rules.

5. Do Not Impersonate or Hide (Ethically)

While some advanced anti-bot measures might necessitate techniques like rotating IP addresses or using headless browsers, prioritize ethical considerations:

* **Transparency:** If possible, be transparent about your scraping activities.
* **Avoid malicious intent:** Do not use scraped data for illegal activities, spam, or competitive disadvantage where it violates the ToS.

6. Data Usage and Copyright

* **Copyright:** Scraped content is often copyrighted. You generally cannot republish, distribute, or monetize scraped data without explicit permission from the copyright holder.
* **Fair Use:** Understand your local "fair use" (or similar) doctrines, but these are often narrow and context-dependent.
* **Personal Use:** Scraping for personal, non-commercial use (e.g., research, personal

archive) is generally less risky but still subject to ToS.

7. Legal Precedents and Risks

The legal landscape for web scraping is evolving and varies by country. Notable cases exist where scraping has been deemed illegal, particularly when it involves:

* Breaching ToS / contract.
* Trespass to chattels (unauthorized access to computer systems causing harm).
* Copyright infringement.
* Data privacy violations (e.g., scraping personal identifiable information (PII) without consent).

In summary, always act responsibly, respectfully, and legally. When in doubt, seek explicit permission from the website owner.

แหล่งที่มา
1. https://github.com/ElSnoMan/pyleniumio ภายใต้ใบอนุญาต (MIT)
2. https://github.com/Constant-Sudo/Aviator_Bot
3. https://github.com/dreamYiZ/AutoSite
4. https://github.com/Scalr/revizor-tests ภายใต้ใบอนุญาต (Apache - 2.0)